

Relazione

Progetto per la sessione d'esame
2025/2026

Autori

Alessandro Catalano
matricola: 334827

Alessio Cavalieri
matricola: 336599

1 ANALISI

Il team si pone come obiettivo lo sviluppo di un videogioco platform a scorrimento orizzontale in stile runner, in cui il giocatore controlla un cubo che deve raggiungere la fine del livello evitando una serie di ostacoli tramite salti temporizzati con precisione, evitando collisioni letali con gli ostacoli che lo costringerebbero a iniziare da capo il livello.

1.1 Requisiti

Funzionalità obbligatorie:

- Il gioco presenta tre livelli preimpostati selezionabili da un menu iniziale. Ciascun livello propone un percorso differente e una difficoltà crescente, con ostacoli disposti in modo studiato per mettere alla prova riflessi e coordinazione del giocatore.
- Ogni livello sarà accessibile solo se i precedenti saranno già stati completati.
- Game loop strutturato: il sistema di gioco è suddiviso in gestione degli input (pressione della barra spaziatrice), aggiornamento dello stato (movimento degli ostacoli, salto del cubo, controllo delle collisioni) e fase di render (visualizzazione degli oggetti nella loro nuova posizione).
- Implementazione di hitbox coerenti e controlli di collisione affidabili tra cubo e ostacoli.
- Il giocatore può eseguire un salto (o più di uno tenendo premuta la barra spaziatrice) per evitare gli ostacoli; il cubo mantiene una velocità di scorrimento costante durante tutta la partita.
- Checkpoint: Possibilità per il giocatore di ricominciare il livello da un punto intermedio prestabilito invece di dover ripartire da capo.
- Ostacoli aggiuntivi: introduzione di elementi che permettono salti doppi, alterano temporaneamente la velocità, le dimensioni o altre caratteristiche del cubo aggiungendo varietà e complessità al gameplay.
- Possibilità di mettere in pausa la partita scegliendo se tornare al menu principale o continuarla.

Funzionalità opzionali

- Colonna sonora: gestione di una colonna sonora durante i livelli e nel menu principale.
- Salvataggio dei livelli completati nel corso di una partita precedente, con conseguente possibilità di continuare la partita da un file di salvataggio.

Challenge principali: Gestione avanzata degli ostacoli: controllo dello spawn e del despawn, mantenimento delle performance, definizione delle hitbox e sincronizzazione degli oggetti con lo scorrimento del livello.

Meccaniche del salto e degli ostacoli aggiuntivi: implementazione del comportamento del cubo, delle modifiche temporanee applicate dagli ostacoli aggiuntivi e della fisica del salto.

Controllo delle collisioni: gestione accurata delle interazioni tra cubo e ostacoli, garantendo al tempo stesso reattività e coerenza visiva.

Ottimizzazione del game loop: mantenimento di un framerate costante, gestione efficiente del rendering e delle operazioni logiche ad ogni frame.

2 Analisi del problema

2.1 Dati di Ingresso del Problema

I dati di ingresso del problema sono le due stringhe non vuote contenenti solo lettere minuscole.

2.2 Dati di Uscita del Problema

I dati di uscita del problema sono:

- La lunghezza della prima stringa, ossia il numero di caratteri che la compongono escluso il carattere terminatore.
- Il risultato dell'inclusione di una stringa nell'altra e viceversa, ossia se si verifica o meno.

2.3 Relazioni Intercorrenti tra i Dati del Problema

Le relazioni intercorrenti tra i dati del problema sono:

- L'inclusione di una stringa nell'altra. Sia Σ un alfabeto finito e non vuoto, e sia $s \in \Sigma^*$ una stringa:

- Prefisso: una stringa $p \in \Sigma^*$ è un prefisso di s se esiste $u \in \Sigma^*$ tale che

$$s = p \cdot u.$$

- Suffisso: una stringa $q \in \Sigma^*$ è un suffisso di s se esiste $v \in \Sigma^*$ tale che

$$s = v \cdot q.$$

- Sottstringa: una stringa $w \in \Sigma^*$ è un infisso (o sottstringa) di s se esistono $u, v \in \Sigma^*$ tali che

$$s = u \cdot w \cdot v.$$

- La lunghezza della prima stringa: essa rappresenta la relazione tra una stringa e un numero naturale che corrisponde al numero di simboli da cui è composta.

Sia Σ un alfabeto finito e non vuoto.

Sia Σ^* l'insieme di tutte le stringhe finite (incluse la stringa vuota) su Σ .

Definiamo la funzione lunghezza:

$$\ell : \Sigma^* \rightarrow \mathbb{N}$$

tale che, per ogni stringa

$$s = a_1 a_2 \dots a_n \quad \text{con } a_i \in \Sigma,$$

vale:

$$\ell(s) = n$$

Ad esempio:

- Se $s = \varepsilon$, allora $\ell(s) = 0$.
- Se $s = a$, allora $\ell(s) = 1$.
- Se $s = \textit{matematica}$, allora $\ell(s) = 10$.

3 Progettazione dell'Algoritmo

3.1 Scelte di Progetto

Le due stringhe contenenti solo caratteri alfabetici minuscoli verranno allocate dinamicamente in quanto non si conosce a priori la loro lunghezza, in modo da evitare limitazioni rispetto alla specifica del problema.

Per la stampa a caratteri giganti della lunghezza della prima stringa e dei messaggi di inclusione in una stringa dell'altra o meno i caratteri utilizzati sono:

- Le cifre da 0 a 9.
- Le lettere "A", "C", "E", "I", "M", "N", "O", "P" e "R" per le stringhe "COMPARE IN" e "NON".

La stampa dei caratteri giganti avviene attraverso un approccio sequenziale noto come "riga per riga". Questo significa che, una volta stampata la prima riga di caratteri giganti, il processo si sposterà automaticamente alla riga successiva, iniziando la stampa dalla posizione di inizio della nuova riga. Questo ciclo si ripete fino a quando non viene visualizzata l'intera sequenza di caratteri giganti, garantendo che ogni riga sia stampata consecutivamente e che l'intero messaggio sia reso visibile.

3.2 Passi dell'Algoritmo

I passi dell'algoritmo per risolvere il problema sono i seguenti:

- Acquisire la prima stringa non vuota e verificare che contenga solo lettere minuscole.
- Acquisire la seconda stringa non vuota e verificare che contenga solo lettere minuscole.
- Calcolare ricorsivamente la lunghezza della prima stringa:
 - Se la stringa non è terminata passare al carattere nella posizione successiva aumentando di 1 il valore della lunghezza.
 - Se la stringa è terminata (si è giunti al carattere '\0') restituire la lunghezza calcolata finora.
- Verificare se la prima stringa è inclusa nella seconda, ad ogni iterazione passando al carattere successivo della prima stringa:
 - Se i caratteri nelle posizioni correnti della prima e della seconda stringa sono uguali passare alla posizione successiva della seconda stringa.

- Se i caratteri nelle posizioni correnti della prima e della seconda stringa non sono uguali tornare al carattere nella prima posizione della seconda stringa.
 - Se il carattere nella posizione corrente della seconda stringa è `\0` allora la seconda stringa è inclusa nella prima.
 - Se si raggiunge il carattere `\0` nella prima stringa senza che si verifichi il caso del punto precedente, la seconda stringa non è inclusa nella prima.
- Verificare se la seconda stringa è inclusa nella prima con lo stesso procedimento del caso precedente.
 - Stampare a caratteri giganti formati da asterischi la lunghezza della prima stringa.
 - Stampare a caratteri giganti formati da asterischi se la prima stringa compare nella seconda o meno, aggiungendo "non" in quest'ultimo caso.
 - Stampare a caratteri giganti formati da asterischi se la seconda stringa compare nella prima o meno, aggiungendo "non" in quest'ultimo caso.

Per evitare ridondanze di codice nei primi due passi dell'algoritmo, è necessario sviluppare un sottoprogramma che si occupi di acquisire una stringa non vuota di lettere minuscole, da utilizzare due volte. Anche per la stampa a caratteri giganti della lunghezza della prima stringa e dell'inclusione di una stringa nell'altra verrà sviluppato un unico sottoprogramma da utilizzare per entrambi i casi (la prima nella seconda e la seconda nella prima).

4 Implementazione dell'Algoritmo

4.1 Implementazione del Codice

```

/*****
/*      Progetto per la sessione d'esame autunnale 2024/2025      */
/*      Insegnamento di Programmazione Procedurale              */
/*                                                                */
/* Programma per il calcolo ricorsivo della lunghezza della      */
/* prima stringa e verifica dell'inclusione di una              */
/* stringa nell'altra e viceversa                                */
/*                                                                */
/*                                                                */
/* Autori: Catalano Alessandro      Matricola:      334827      */
/*      Grazia Diego                Matricola:      336599      */
*****/

/*****/
/* Inclusione delle librerie */
/*****/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*****/
/* dichiarazione delle funzioni */
/*****/

char* acquisizione_stringa(char *);
int   calcolo_lunghezza_stringa(char *);
int   inclusione_stringa(char *,
                        char *);
void  stampa_lunghezza(int);
void  stampa_risultato_inclusione(int,
                                int,
                                int);
void  stampa_testo_gigante(char*);

/*****/
/* definizione delle funzioni */
/*****/

/* definizione della funzione main */
int main(void)
{
    /* dichiarazione delle variabili locali alla funzione */
    char *stringa1,          /* input: prima stringa di lettere minuscole */
          *stringa2;        /* input: seconda stringa di lettere minuscole */
    int  lunghezza_stringa, /* output: lunghezza della prima stringa */

```

```

        inclusione_stringa_1_in_2, /* output: risultato inclusione della
                                   prima stringa nella seconda */
        inclusione_stringa_2_in_1; /* output: risultato inclusione della
                                   stringa nella prima */

lunghezza_stringa = 0;

/* acquisire la prima stringa di lettere minuscole */
stringa1 = acquisizione_stringa ("prima");

/* acquisire la seconda stringa di lettere minuscole */
stringa2 = acquisizione_stringa ("seconda");

/* calcolare ricorsivamente la lunghezza della prima stringa */
lunghezza_stringa = calcolo_lunghezza_stringa(stringa1);

/* verificare se la prima stringa è inclusa nella seconda */
inclusione_stringa_1_in_2 = inclusione_stringa(stringa1,
                                              stringa2);

/* verificare se la seconda stringa è inclusa nella prima */
inclusione_stringa_2_in_1 = inclusione_stringa(stringa2,
                                              stringa1);

printf("\n");

/* stampare la lunghezza della prima stringa */
stampa_lunghezza(lunghezza_stringa);

printf("\n");

/* stampare il risultato dell'inclusione della prima stringa nella seconda */
stampa_risultato_inclusione(1,
                           2,
                           inclusione_stringa_1_in_2);

printf("\n");

/* stampare il risultato dell'inclusione della seconda stringa nella prima */
stampa_risultato_inclusione(2,
                           1,
                           inclusione_stringa_2_in_1);

return 0;
}

/* definizione della funzione per l'acquisizione delle due stringhe */
char* acquisizione_stringa(char *messaggio) /* input: messaggio da stampare */
{

```

```

/* dichiarazione delle variabili locali alla funzione */
char *stringa;          /* output: stringa da acquisire */
int  acquisizione_errata, /* lavoro: esito complessivo dell'acquisizione */
     lung_stringa,        /* lavoro: lunghezza della stringa acquisita */
     i;                  /* lavoro: indice caratteri stringa */

/* acquisizione della stringa */
do
{
    printf("Inserisci la %s sequenza di sole lettere minuscole: ",
           messaggio);
    scanf("%ms",
          &stringa);

    lung_stringa = strlen(stringa);
    acquisizione_errata = 0;

    /* verifica della presenza di sole lettere minuscole */
    for(i = 0;
        i < lung_stringa;
        i++)
    {
        if(stringa[i] < 'a' || stringa[i] > 'z')
            acquisizione_errata++;
    }

    if (acquisizione_errata > 0)
        printf("Sequenza non accettabile! \n");
}
while (acquisizione_errata > 0);

return stringa;
}

/* definizione della funzione per il calcolo ricorsivo della lunghezza della prima stringa */
int calcolo_lunghezza_stringa(char *stringa1) /* input: prima stringa */
{
    int lunghezza_stringa; /* output: lunghezza della prima stringa*/

    /* verificare se la prima stringa è terminata */
    if (*stringa1 == '\0')
    {
        lunghezza_stringa = 0;
    }
    else
    {
        lunghezza_stringa = 1 + calcolo_lunghezza_stringa(stringa1 + 1);
    }
}

```

```

    return lunghezza_stringa;
}

/* definizione della funzione per verificare l'inclusione di una stringa nell'altra */
int inclusione_stringa(char *stringa_inclusa, /* input: stringa di cui
                                             verificare l'inclusione nell'altra */
                      char *stringa_includente) /* input: stringa in cui verificare
                                             l'inclusione nell'altra stringa */
{
    int i, /* lavoro: indice dei caratteri di stringa_inclusa */
        k, /* lavoro: indice dei caratteri di stringa_includente */
        inclusione; /* outpt: risultato dell'inclusione di stringa_inclusa in stringa_includente*/

    /* verificare se stringa_inclusa è inclusa in stringa_includente */
    for (k = 0, inclusione = 0; stringa_includente[k] != '\0' && inclusione == 0; k++)
    {
        for (i = 0;
             stringa_inclusa[i] != '\0' && stringa_includente[k + i] == stringa_inclusa[i];
             i++);

        if (stringa_inclusa[i] == '\0')
            inclusione = 1;
    }

    return inclusione;
}

/* definizione della funzione per stampare la lunghezza della prima stringa */
void stampa_lunghezza(int lunghezza_stringa) /* output: lunghezza prima stringa*/
{
    char numero[20]; /* lavoro: stringa per stampare la lunghezza della prima stringa */

    /* convertire la lunghezza della prima stringa in una stringa */
    sprintf(numero,
            "%d",
            lunghezza_stringa);

    stampa_testo_gigante(numero);
}

/* dichiarazione della funzione per stampare il risultato
dell'inclusione di una stringa nell'altra */
void stampa_risultato_inclusione(int stringa_inclusa, /* input: stringa di cui è stata
                                                       verificata l'inclusione nell'altra */
                                int stringa_includente, /* input: stringa in cui è stata
                                                       verificata l'inclusione dell'altra */
                                int inclusione_stringa) /* output: risultato dell'inclusione */
{
    char testo[20]; /* lavoro: stringa per stampare risultato dell'inclusione

```

```

        di una stringa nell'altra*/

/* assegnare a testo il messaggio corretto da stampare in base al risultato
   dell'inclusione di stringa_inclusa in stringa_includente */
if (inclusione_stringa == 0)
    snprintf(testo, sizeof(testo),
        "%d non compare in %d",
        stringa_inclusa,
        stringa_includente);
else
    snprintf(testo, sizeof(testo),
        "%d compare in %d",
        stringa_inclusa,
        stringa_includente);

stampa_testo_gigante(testo);
}

/* dichiarazione della funzione per stampare la lunghezza della prima stringa
   e il risultato dell'inclusione di una stringa nell'altra */
void stampa_testo_gigante(char *testo) /* output: testo da stampare con caratteri giganti */
{
    int i,                /* lavoro: indice della posizione del messaggio da stampare */
        indice,          /* lavoro: indice del carattere da stampare */
        riga;            /* lavoro: riga da stampare */
    char carattere;        /* carattere da stampare */
    char *caratteri[19][5] = /* lavoro: array che contiene tutti i caratteri
                               necessari alla stampa */

    {
        {
            "*****",
            "*   *",
            "*   *",
            "*   *",
            "*****"
        }, /* 0 */

        {
            " *  ",
            " ** ",
            "* *  ",
            " *  ",
            "*****"
        }, /* 1 */

        {
            "*****",
            " *   ",
            "*****",
            "*   "
        }
    }
}

```

```

    "*****"
}, /* 2 */

{
    "*****",
    "    *",
    "*****",
    "    *",
    "*****"
}, /* 3 */

{
    "    * ",
    "   ** ",
    " * * ",
    "*****",
    "    * "
}, /* 4 */

{
    "*****",
    "*      ",
    "*****",
    "    *",
    "*****"
}, /* 5 */

{
    "*****",
    "*      ",
    "*****",
    "*    *",
    "*****"
}, /* 6 */

{
    "*****",
    "    *",
    "    *",
    "    *",
    "    *"
}, /* 7 */

{
    "*****",
    "*    *",
    "*****",
    "*    *",
    "*****"
}, /* 8 */

```

```

{
    "*****",
    " *  *",
    "*****",
    "   *",
    "*****"
}, /* 9 */

```

```

{
    " *  ",
    " * * ",
    "*****",
    "*   *",
    "*   *"
}, /* A */

```

```

{
    "*****",
    "*      ",
    "*      ",
    "*      ",
    "*****"
}, /* C */

```

```

{
    "*****",
    "*      ",
    "*****",
    "*      ",
    "*****"
}, /* E */

```

```

{
    "*****",
    " *  ",
    " *  ",
    " *  ",
    "*****"
}, /* I */

```

```

{
    "*   *",
    "**  **",
    "* * *",
    "*   *",
    "*   *"
}, /* M */

```

```

{

```

```

        "*   *",
        "***  *",
        "* *  *",
        "*   **",
        "*   *"
    }, /* N */

    {
        "*****",
        "*   *",
        "*   *",
        "*   *",
        "*****"
    }, /* O */

    {
        "*****",
        "*   *",
        "*****",
        "*   ",
        "*   "
    }, /* P */

    {
        "*****",
        "*   *",
        "*****",
        "* *  ",
        "*   *"
    }, /* R */
};

/* stampare il messaggio con caratteri giganti con cinque caratteri di altezza e larghezza */
for (riga = 0; riga < 5; riga++)
{
    for (i = 0; testo[i] != '\0'; i++)
    {
        indice = -1;
        carattere = testo[i];

        if (carattere >= '0' && carattere <= '9')
        {
            indice = carattere - '0';
        }
        else
        {
            switch (carattere)
            {
                case 'a':
                    indice = 10; break;
            }
        }
    }
}

```

```

        case 'c':
            indice = 11; break;
        case 'e':
            indice = 12; break;
        case 'i':
            indice = 13; break;
        case 'm':
            indice = 14; break;
        case 'n':
            indice = 15; break;
        case 'o':
            indice = 16; break;
        case 'p':
            indice = 17; break;
        case 'r':
            indice = 18; break;
        default:
            indice = -1; break; /* caso di carattere non previsto*/
    }
}
if (indice >= 0)
    printf("%s ", caratteri[indice][riga]);
else
    printf("      "); /* spazi per caratteri non previsti */
}
printf("\n");
}
}

```

4.2 Implementazione del makefile

```
lunghezza_inclusione: lunghezza_inclusione.c makefile
    gcc -ansi -Wall -O lunghezza_inclusione.c -o lunghezza_inclusione
pulisci:
    rm -f lunghezza_inclusione.o
pulisci_tutto:
    rm -f lunghezza_inclusione lunghezza_inclusione.o
```

5 Testing del Programma

5.1 12 Esempi di Input e Output

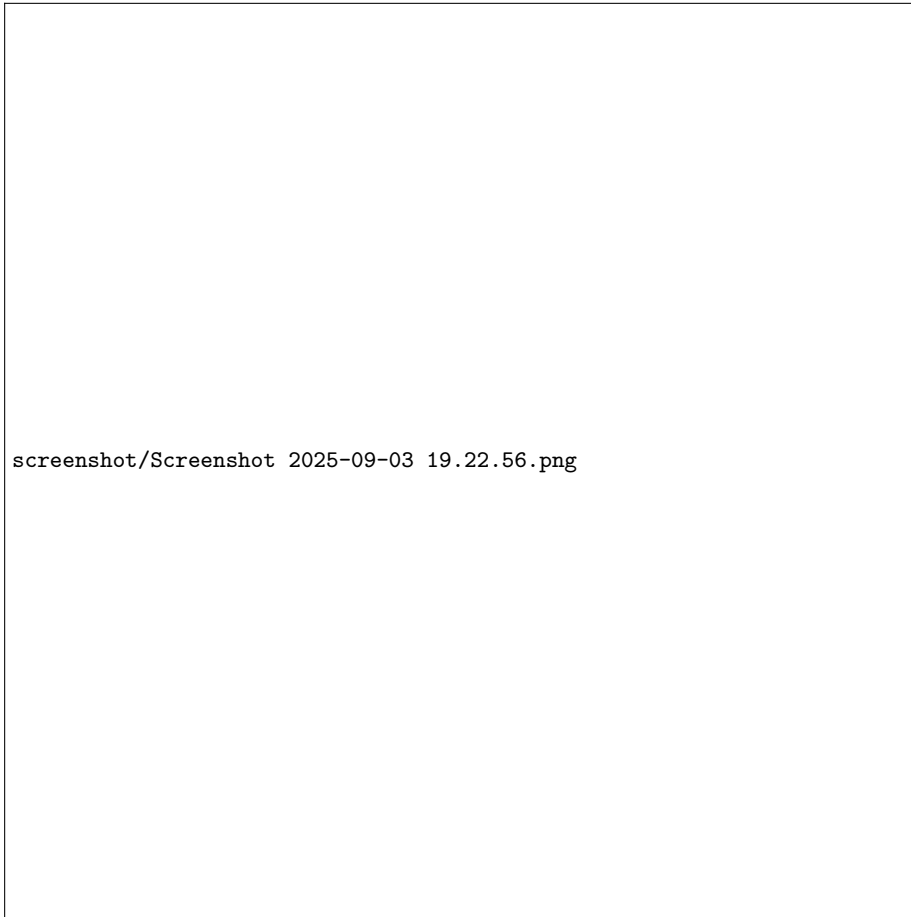
- Esempio 1:



- Esempio 2:

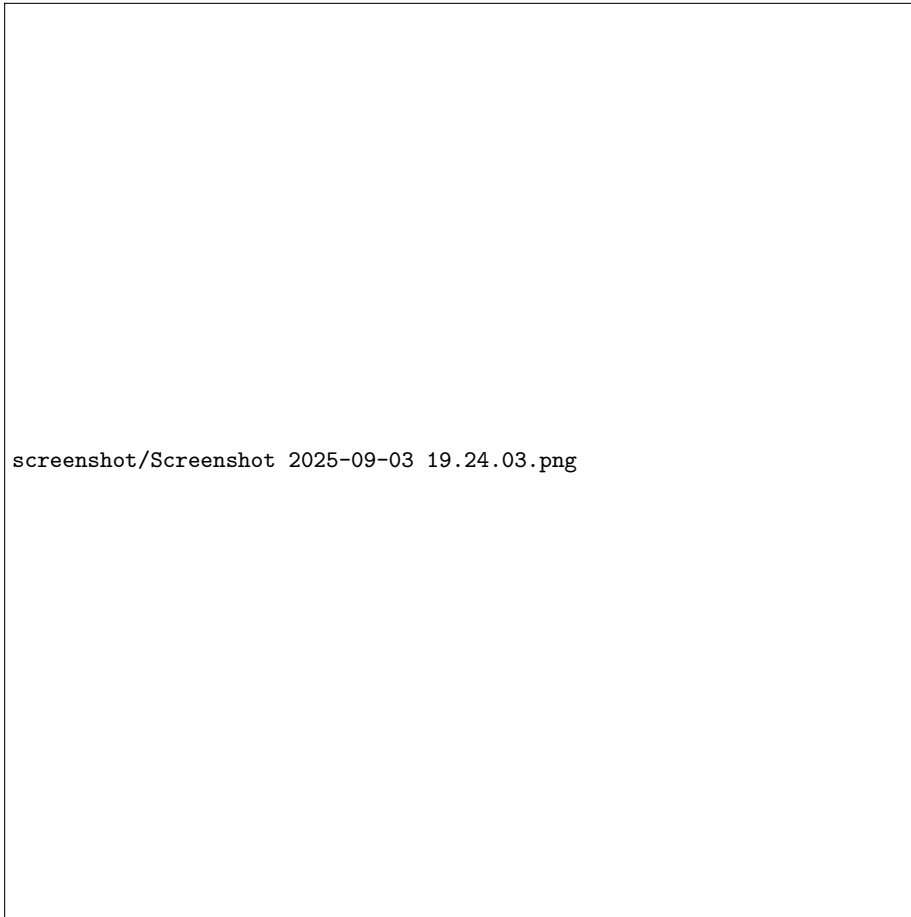


- Esempio 3:

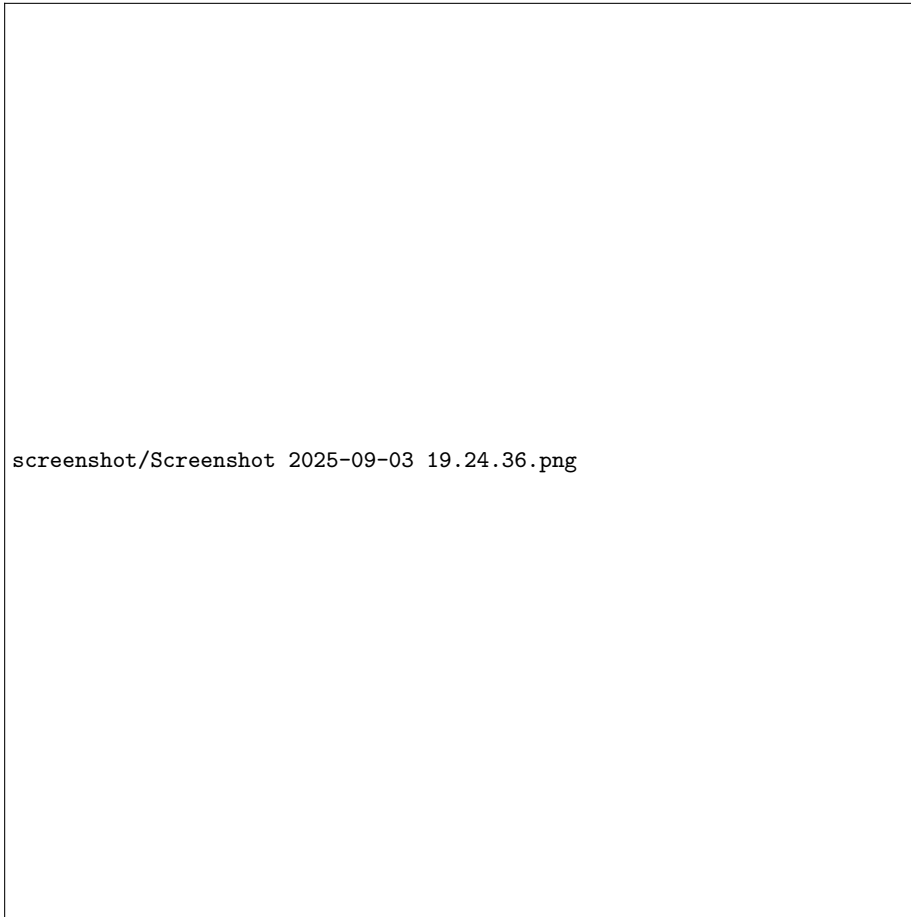


screenshot/Screenshot 2025-09-03 19.22.56.png

- Esempio 4:



- Esempio 5:

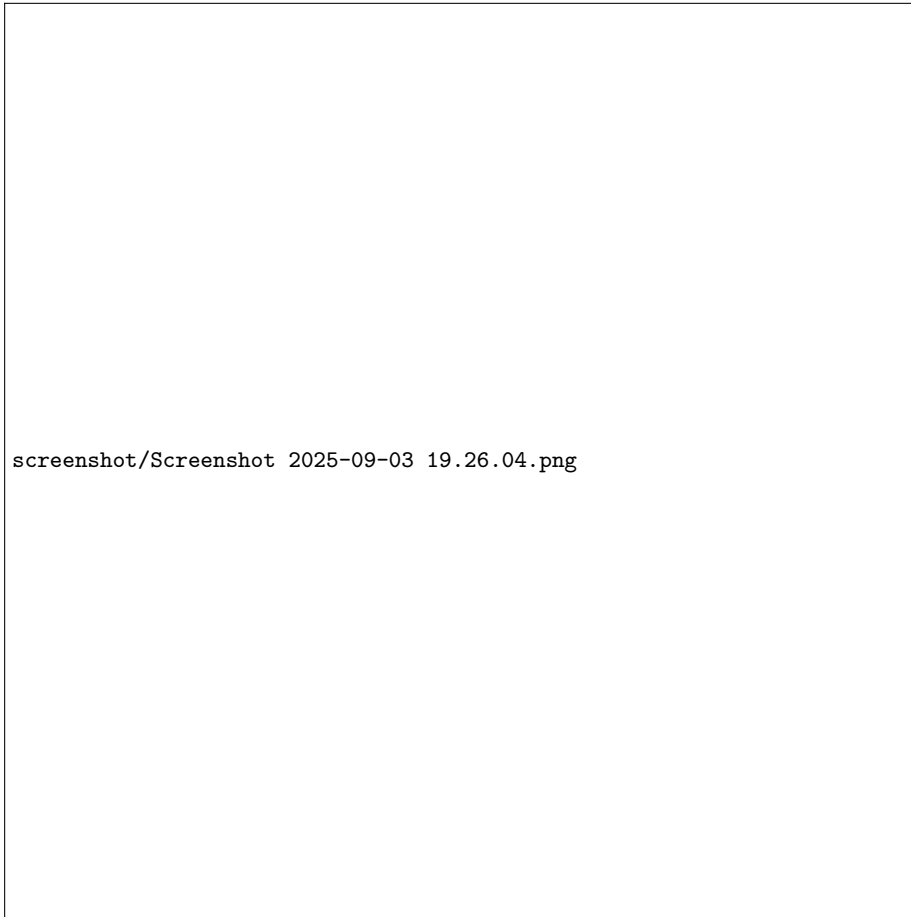


screenshot/Screenshot 2025-09-03 19.24.36.png

- Esempio 6:



- Esempio 7:

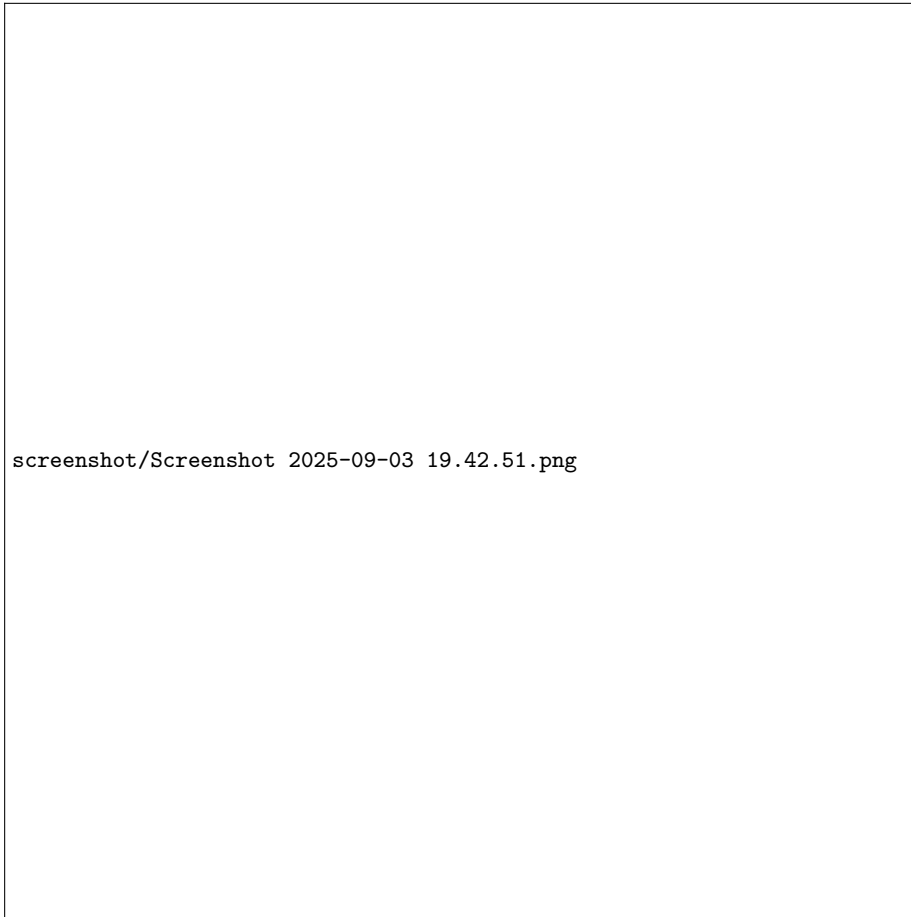


screenshot/Screenshot 2025-09-03 19.26.04.png

- Esempio 8:

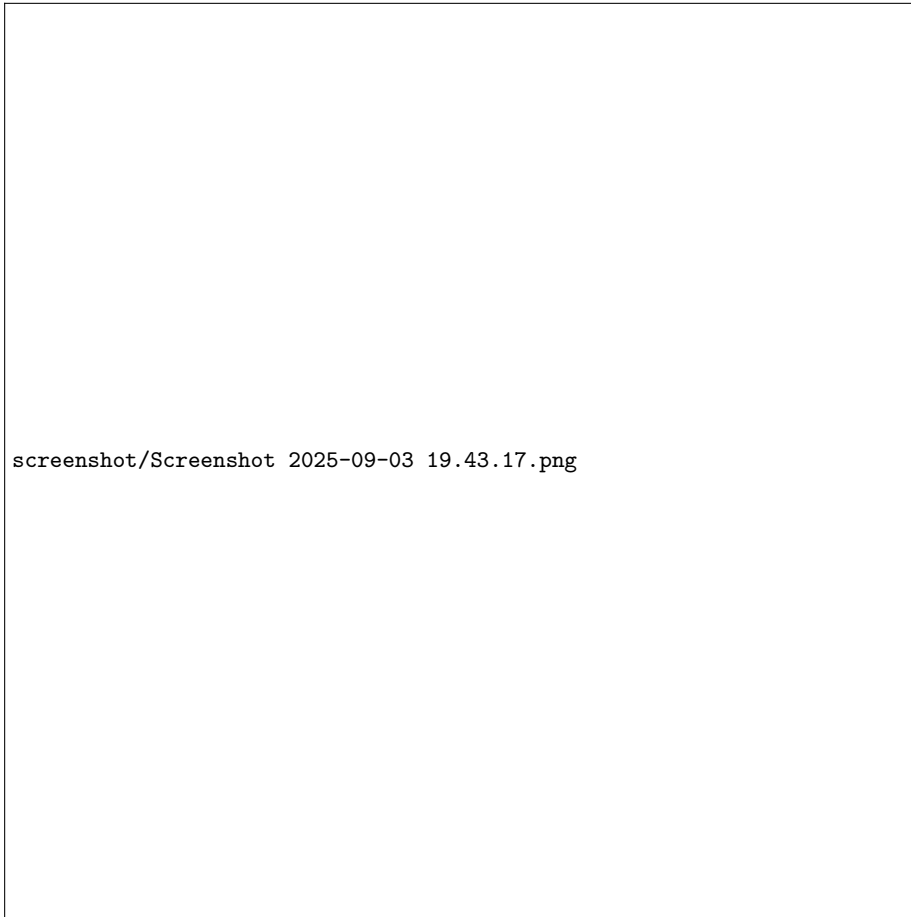


- Esepio 9:



screenshot/Screenshot 2025-09-03 19.42.51.png

- Esempio 10:

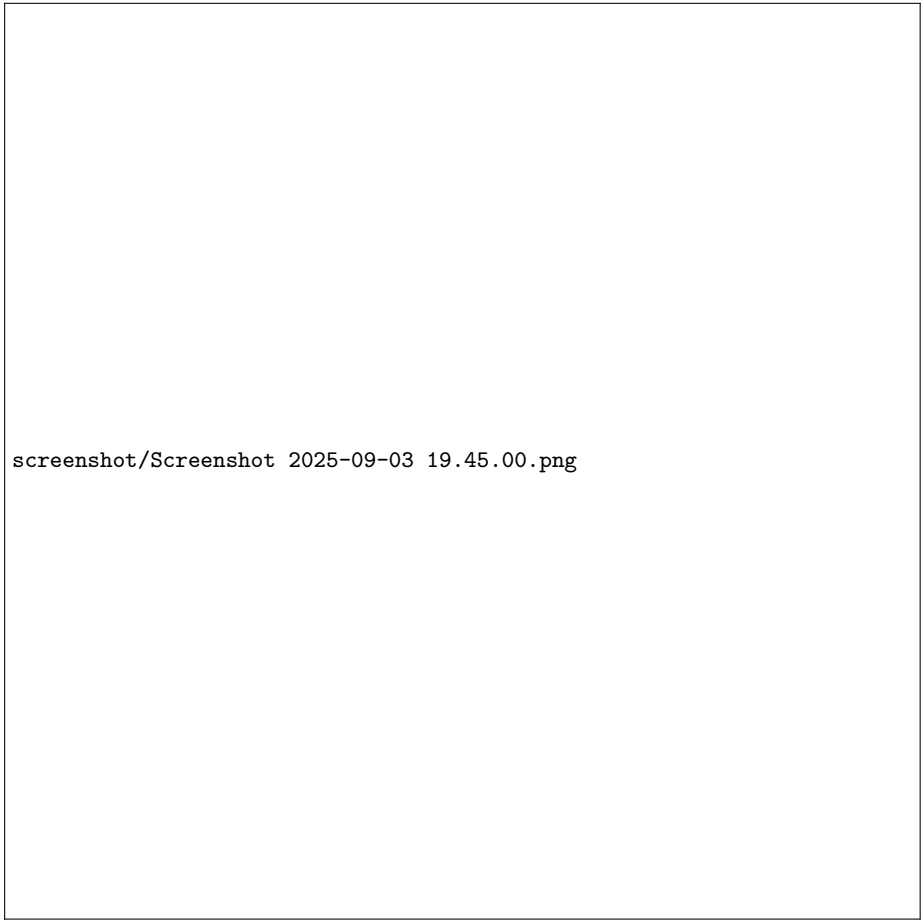


- Esempio 11:



screenshot/Screenshot 2025-09-03 19.43.39.png

- Esempio 12:

A large rectangular box containing a screenshot of a document page. The screenshot shows a white background with a single line of text in a monospaced font. The text is a file path: 'screenshot/Screenshot 2025-09-03 19.45.00.png'.

screenshot/Screenshot 2025-09-03 19.45.00.png

6 Verifica del Programma

6.1 Brano di Codice Scelto

Il brano di codice scelto è la funzione ricorsiva per calcolare la lunghezza della prima stringa.

```
int calcolo_lunghezza_stringa(char *stringa1) /* input: prima stringa */
{
    int lunghezza_stringa; /* output: lunghezza della prima stringa*/

    /* verificare se la prima stringa 'e terminata */
    if (*stringa1 == '\0')
    {
        lunghezza_stringa = 0;
    }
}
```

```

else
{
    lunghezza_stringa = 1 + calcolo_lunghezza_stringa(stringa1 + 1);
}
return lunghezza_stringa;
}

```

Il brano di codice scelto può essere riscritto nella seguente maniera (per semplificare `lung_stringa1 = l`, `stringa1 = s` e `calcolo_lunghezza_stringa = calc_l_s`):

```

int calc_l_s(char *s) /* input: prima stringa */
{
    int l; /* output: lunghezza della prima stringa*/

    /* verificare se la prima stringa è terminata */
    if (*s == '\0')
    {
        l = 0;
    }
    else
    {
        l = 1 + calc_l_s (s + 1);
    }
    return l;
}

```

6.2 Proprietà da Verificare

La proprietà che vogliamo verificare è che per ogni sequenza di caratteri s risulta:

$$calc_l_s(s) = |val(s)|,$$

dove $|val(s)|$ è la lunghezza della stringa s al netto del carattere di terminazione $'\0'$.

6.3 Svolgimento

La proprietà si può dimostrare procedendo per induzione:

- **Passo base:** consideriamo la stringa vuota, ossia $val(s) = \varepsilon$. Poiché $*s = '\0' \Rightarrow calc_l_s(s) = 0$, l'asserto è vero se la stringa è vuota.
- **Passo induttivo:** supponiamo ora che $n > 0$ e che la proprietà sia valida per ogni stringa di lunghezza minore di n . Scriviamo allora $val(s) = a \circ s'$, con a primo carattere e $|s'| = n - 1$. La funzione esegue il ramo ricorsivo e calcola:
$$calcolo_lunghezza_stringa(s) = 1 + calcolo_lunghezza_stringa(s + 1).$$

Per ipotesi induttiva sappiamo che

$$calcolo_lunghezza_stringa(s + 1) = |s'|.$$

Segue quindi:

$$calcolo_lunghezza_stringa(s) = 1 + |s'| = |a \circ s'| = |val(s)|.$$